ENTERPRISE AGENTIC UNDERWRITING PIPELINE

Technical documentation

System design, multi-agent workflow, ML models, explainability, fairness, MCP tools and verified findings.

Basis of this document. Every statement was checked against the supplied project (`underwriting_engine`): all Python modules, `README.md`, `requirements.txt`, the generated dataset, the decision log and the trained artefacts. Where the shipped artefacts behave differently from what the code is designed to do, both are stated.

Item	Detail
Project folder	`underwriting_engine/` (agents, FastAPI app, MCP server, Flask dashboard, training scripts)
Services	FastAPI on port 8000; MCP server on 8001; Flask dashboard on 5000; optional Ollama on 11434
Model stack	XGBoost (default probability), Isolation Forest (anomalies), SHAP TreeExplainer (attributions)
Data	Synthetic only: 2,000 generated applicants, 1,836 with consent
Intended use	Personal learning and skill-building project. Not for real credit decisions.

Contents

1. Problem statement and project overview
2. High-level architecture
3. End-to-end underwriting flow
4. Agent-by-agent explanation
5. Synthetic data and model training
6. Feature dictionary
7. Decision policy and score interpretation
8. Fairness and audit utilities
9. MCP server and tools
10. REST API reference
11. Dashboard
12. Running the project
13. Verified findings and recommended fixes
14. Security, privacy and responsible use
15. Glossary
16. Conclusion

1. Problem statement and project overview

Problem statement

Traditional underwriting relies mainly on bureau scores and historical financial records. This excludes valuable behavioural and digital signals, so applicants whose creditworthiness is not captured by bureau data can be assessed poorly.

This project proposes an AI-powered dynamic underwriting engine that combines traditional financial data with **consented** alternative data to provide:

- Dynamic credit-risk scoring on a 0–100 scale.
- Explainable decisions (SHAP attributions and a plain-language letter).
- Fraud and anomaly detection.
- Fairness auditing using the 80% rule across gender and age groups.
- Continuous re-scoring as behaviour changes.
- Consent verification aligned to India's DPDP Act 2023.

Overview

The Enterprise Agentic Underwriting Pipeline is a prototype credit-risk assessment system. It accepts applicant information with a consent flag, derives alternative-data indicators, runs fraud and credit-risk models, applies a deterministic policy, explains the result with SHAP and a plain-language letter, performs a rules-based self-review and logs the outcome. FastAPI exposes the workflow over REST, a separate MCP server exposes selected data and model operations as tools, and a Flask page provides a demo console.

Conventional inputs (bureau-style score, income, debt-to-income ratio, existing loans) are combined with behavioural signals (UPI volume, BNPL repayment, utility punctuality, telecom recharge regularity, gig-income stability, e-commerce behaviour, device changes, digital-footprint consistency).

> **Current state of the shipped model**
>
> The pipeline structure runs end to end, but the supplied dataset contains **one default in 2,000 rows** (0.05%). The trained XGBoost model therefore returns the **same probability of default (0.00068) and credit-risk score (99.9) for every input**, including the 'suspicious' demo profile. Decisions only differ because of the fraud rules and the Isolation Forest.
>
> Consequences: scores, SHAP drivers and fairness results currently carry no information. Section 5 explains the cause and Section 13 gives the fix.

Objectives

- Block processing when the consent flag is false.
- Use alternative signals alongside traditional financial features.
- Detect unusual applications with an Isolation Forest and transparent heuristic rules.
- Estimate probability of default with XGBoost and convert it to a 0–100 credit-risk score.
- Explain each prediction with SHAP and an Ollama-generated (or fallback) decision letter.
- Check policy consistency and protected-term leakage before returning a result.

- Support fairness auditing, what-if re-scoring, MCP tool access and decision history.

2. High-level architecture

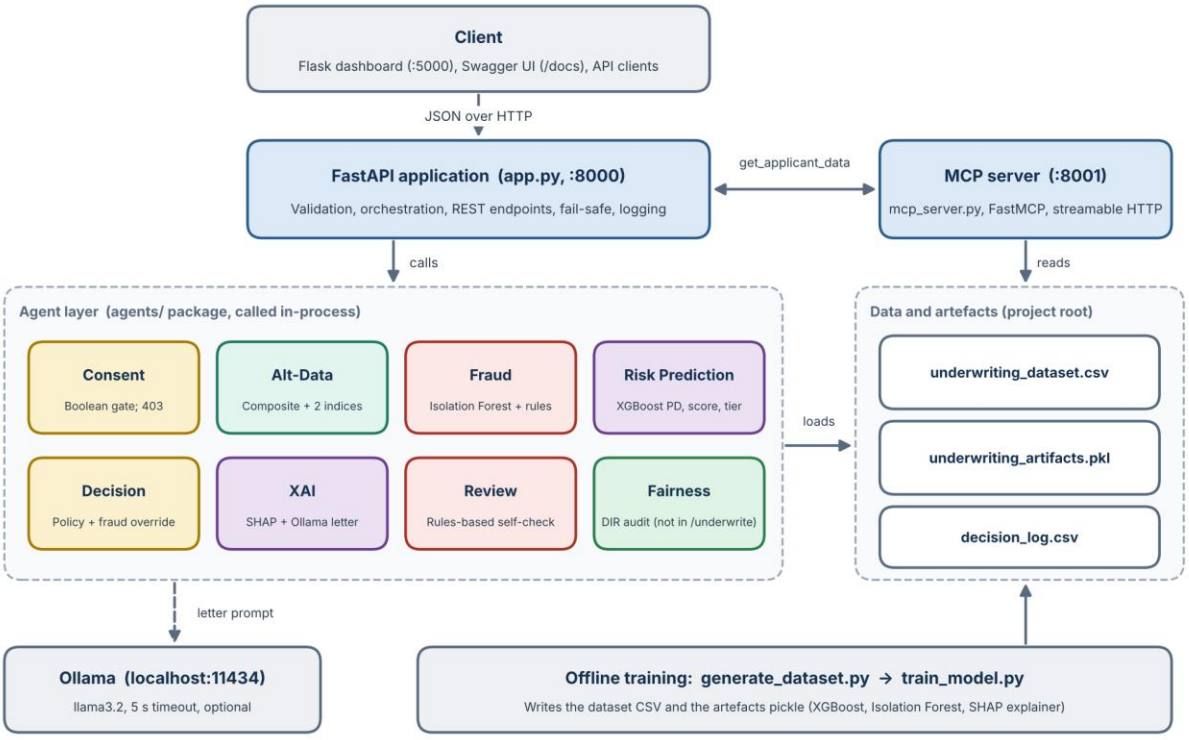


Figure 1. Logical architecture. FastAPI calls the agents in-process; the MCP server is a separate process reached over HTTP.

Components and files

Component	File(s)	Responsibility
FastAPI application	`app.py`	Loads artefacts at start-up, validates requests, orchestrates agents, applies the review fail-safe, writes the log and serves the REST API.
Agent layer	`agents/*.py`	Eight small classes: consent, alternative data, fraud, risk prediction, explainability, decision, fairness and review.
MCP server	`mcp_server.py`	FastMCP server exposing six tools and one resource over streamable HTTP at `127.0.0.1:8001/mcp`.
Dashboard	`dashboard_flask.py`	Single-page Flask console (port 5000) with four preset applicants and sliders; posts to `http://localhost:8000/api/v1/underwrite`.
Training scripts	`generate_dataset.py`, `train_model.py`	Create the synthetic dataset; train and serialise the models into `underwriting_artifacts.pkl`.
Artefacts	`underwriting_artifacts.pkl`	joblib dictionary: `model`, `isolation_forest`, `explainer`, `feature_names`, `anomaly_columns`.

Component	File(s)	Responsibility
Dataset	`underwriting_dataset.csv`	2,000 generated rows; used for MCP lookups, fairness audits and training.
Decision log	`decision_log.py`, `decision_log.csv`	Append-only CSV with seven fields; history lookup by applicant ID.
Standalone audit	`fairness_audit.py`	Command-line fairness report with its own approval rule (see Section 8).
Local LLM	Ollama, `llama3.2`	Optional; drafts the decision letter with a 5-second timeout.

3. End-to-end underwriting flow

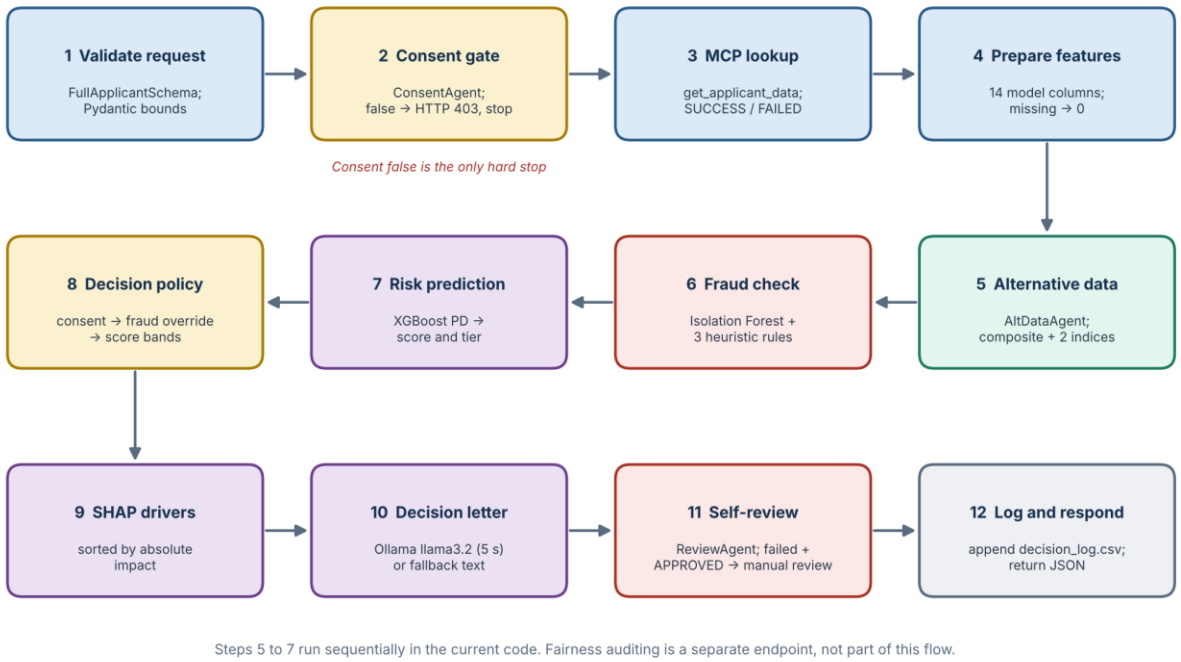


Figure 2. Request flow for POST /api/v1/underwrite.

Steps follow the order of the code in `app.py`:

1. **Validation.** FastAPI parses the body into `FullApplicantSchema`. Pydantic enforces bounds (score 300–850, rates 0–1, non-negative counts). `num_existing_loans` and `mobile_wallet_tx_freq` are optional and default to 0.
2. **Consent gate.** `ConsentAgent.verify_consent()` returns HTTP 403 with a message when `consent_given` is false. This is the only hard stop.
3. **MCP lookup.** The app opens a new streamable-HTTP session and calls `get_applicant_data`. The status is recorded as `SUCCESS` or `FAILED`; on failure the pipeline continues. The returned record is included in the response as diagnostic data only.
4. **Feature preparation.** `prepare_feature_df()` builds a one-row DataFrame in the saved feature order. Missing model features are filled with 0. The function also prints the feature list and row to the console (debug output).

5. **Alternative data, fraud and risk.** Three sequential calls on the request payload: `AltDataAgent.process_signals`, `FraudAgent.evaluate_fraud_risk`, `RiskPredictionAgent.predict_risk`. They are not run concurrently.
6. **Decision policy.** `DecisionAgent.render_decision()` applies consent, then the fraud override, then score bands.
7. **Explainability.** `XAIAgent` computes SHAP attributions and asks Ollama for a letter, using the decision from the previous step.
8. **Self-review.** `ReviewAgent.review()` runs four checks. If it fails and the decision is `APPROVED`, the app replaces the decision with `FLAGGED_FOR_MANUAL_REVIEW`. The letter is **not** regenerated, so it can still say 'approved' while the returned decision is manual review.
9. **Log and response.** One row is appended to `decision_log.csv`. The JSON response contains `applicant_id`, `mcp`, `decision`, `decision_reason`, `credit_risk_score`, `risk_tier`, `default_probability`, `alt_data_metrics`, `fraud_assessment`, the top four SHAP drivers, `explanation_letter` and `self_check`.

4. Agent-by-agent explanation

4.1 ConsentAgent (agents/consent_agent.py)

`verify_consent(consent_given, applicant_id)` returns `verified`, `applicant_id`, `status_code` (403 or 200) and a message. The regulation label (default 'Indian DPDP Act 2023') is a constructor argument used only in message text.

> **Limitation**
> A Boolean flag is a technical gate. It does not prove that consent was informed, specific, recorded, withdrawable or legally sufficient, so the project should not be described as DPDP-compliant on this basis.

4.2 AltDataAgent (agents/alt_data_agent.py)

`process_signals(raw_data)` returns three summaries, all informational (the model consumes the raw columns, not these values):

- `alt_composite_score` = UPI health (volume ÷ 50, capped at 1) × 25 + BNPL repayment × 25 + telecom regularity × 25 + gig stability × 25, rounded to one decimal (0–100).
- `financial_footprint_index` = mean of BNPL repayment and utility punctuality (two decimals).
- `digital_reliability_index` = mean of telecom regularity and digital-footprint consistency (two decimals).

The agent indexes the dictionary directly, so a missing key raises `KeyError`.

4.3 FraudAgent (agents/fraud_agent.py)

Two mechanisms are combined. The Isolation Forest (100 trees, contamination 0.05, trained on `digital_footprint_consistency`, `device_change_frequency` and `ecommerce_return_rate`) gives a label (1 or −1) and a decision-function score. Three rules add flags:

- `HIGH_DEVICE_TURNOVER`: device changes greater than 3.

- SUSPICIOUS_ECOM_RETURN_PATTERN: return rate above 0.60.
- INCONSISTENT_IDENTITY_FOOTPRINT: digital-footprint consistency below 0.40.

is_anomalous is true when the forest predicts −1 or at least two flags fire. fraud_risk_level is CRITICAL with two or more flags, ELEVATED when anomalous for another reason, otherwise LOW. Outputs are cast to native Python types so FastAPI can serialise them.

4.4 RiskPredictionAgent (agents/risk_prediction_agent.py)

Selects the saved feature order, reads the class-1 probability from XGBoost as default_probability (four decimals) and computes credit_risk_score = (1 − PD) × 100 (one decimal). Higher is better. Tiers: 75 and above TIER_1_EXCELLENT; 60–74.9 TIER_2_GOOD; 45–59.9 TIER_3_MODERATE; below 45 TIER_4_HIGH_RISK.

4.5 DecisionAgent (agents/decision_agent.py)

Evaluated in this order: no consent → REJECTED_NON_COMPLIANT; anomalous or CRITICAL → FLAGGED_FOR_MANUAL_REVIEW (reason lists the triggered flags, or 'ML Anomaly'); score ≥ 60 → APPROVED; score ≥ 45 → CONDITIONAL_APPROVAL; otherwise DECLINED. The reason for conditional approval mentions a lower credit line, but no credit-limit calculation exists in the code.

4.6 XAIAgent (agents/xai_agent.py)

compute_shap_importance() normalises the SHAP output across library versions (list per class, 2-D, or 3-D arrays), checks that the length equals the feature count, and returns feature, shap_value (four decimals) and raw_value sorted by absolute SHAP value. generate_explanation_letter() sends a prompt containing the decision, score and top three drivers to POST /api/generate on Ollama (llama3.2, stream: false, 5-second timeout).

If Ollama is unreachable or returns a non-200 status, a fixed template is used. Any outcome other than APPROVED is described as **DECLINED** with a 30-day human re-evaluation statement, which is wrong for conditional approval and manual review. The template reads top_features[0], so an empty driver list would raise an error.

4.7 FairnessAgent (agents/fairness_agent.py)

audit(df) keeps consented rows, predicts PD, converts to scores and marks approval as score ≥ approval_score (default 60, matching the live policy). For gender (reference Male) and age_group (reference 25_50) it reports approval_rate, disparate_impact_ratio, demographic_parity_difference and passes_80_percent_rule (threshold 0.80). Gender and age group are excluded from the model features and used only here. If the reference group is absent, the code falls back to a rate of 1.0, which would produce misleading ratios.

4.8 ReviewAgent (agents/review_agent.py)

Rules-based and auditable. It returns self_check_passed, issues and a note. The four checks:

- The decision matches the score bands (manual review and non-compliant outcomes are always accepted).
- An anomalous applicant is not APPROVED.
- No SHAP driver name contains gender, age_group, age, religion, caste or ethnicity.

- The letter contains the first word of the decision, ‘declined’ or ‘approved’, and contains the score as text (for example `99.9`).

Because the same thresholds are used by `DecisionAgent`, check 1 acts as a drift guard rather than a live control. The substring checks on the letter are weak: a letter can pass while misdescribing the outcome.

5. Synthetic data and model training

generate_dataset.py

`generate_underwriting_data(num_samples=2000, seed=42)` samples identifiers (`APP_1000` onwards), gender, age group and consent (92% true), then traditional and alternative variables from normal, log-normal, beta, Poisson and negative-binomial distributions. A latent risk value `z` combines the signals with Gaussian noise (σ = 0.8); `default_prob = sigmoid(z)`; `default_flag = 1` when `default_prob > 0.45`.

train_model.py

- Reads the dataset and keeps consented rows (falls back to all rows if fewer than five rows or only one class).
- Uses 14 explicit `FEATURE_COLUMNS`; gender and age group are excluded.
- 80/20 train–test split with `random_state=42`; stratified only when the rarer class has at least two rows.
- XGBClassifier: 150 trees, depth 4, learning rate 0.05, subsample 0.8, column sample 0.8, log-loss metric.
- Prints ROC-AUC and a classification report (0.45 cut-off) only when the test set has both classes.
- Fits `IsolationForest(n_estimators=100, contamination=0.05)` on the three anomaly columns, builds `shap.TreeExplainer`, and saves everything with joblib.

What the supplied artefacts actually contain

Check	Result
Rows / consented rows	2,000 / 1,836
`default_flag = 1`	1 of 2,000 (0.05%); 1 of 1,836 among consented rows
Latent risk `z`	Mean −4.6, range −10.5 to +0.1; mean sigmoid(z) is 2.5%, and only one row exceeds 0.45
Stratified split	Disabled (rarer class has one row); the test set most likely has no defaults, so no ROC-AUC is printed
Model output, all consented rows	PD 0.00068 and score 99.9 for every row; 100% would be ‘approved’ by score
Dashboard presets (APP_CUSTOM, 1042, 2091, 3310)	All score 99.9; Isolation Forest flags only APP_3310 as anomalous
Decision log (4 rows)	All scores 99.9; APP_3310 is `FLAGGED_FOR_MANUAL_REVIEW` with fraud level `CRITICAL` via the rules

The cause is the generator: the latent expression has no positive offset, so it is strongly negative for nearly everyone, and the hard 0.45 cut-off then produces almost no positive labels. The model effectively learned a constant. Even with a healthy label, metrics on this data would mostly reflect the generator's own formula, not real borrower behaviour.

6. Feature dictionary

The first fourteen rows are the model inputs, in training order. 'API check' is the Pydantic constraint in `FullApplicantSchema`.

Feature	Meaning	API check	Also used by
`traditional_score`	Bureau-style credit score	300–850	Model
`monthly_income_inr`	Monthly income in rupees	≥ 0	Model
`debt_to_income_ratio`	Debt relative to income	0–1	Model
`num_existing_loans`	Number of existing loans	optional, default 0	Model
`upi_monthly_tx_volume`	UPI transactions per month	integer ≥ 0	Model, AltData composite
`bnpl_repayment_rate`	BNPL repayment regularity	0–1	Model, AltData
`utility_pay_punctuality`	On-time utility payments	0–1	Model, AltData
`telecom_recharge_regularity`	Regularity of recharges	0–1	Model, AltData
`ecommerce_monthly_spend`	Monthly e-commerce spend (₹)	≥ 0	Model
`ecommerce_return_rate`	Share of purchases returned	0–1	Model, Fraud (model and rule)
`gig_income_stability_index`	Stability of gig income	0–1	Model, AltData
`digital_footprint_consistency`	Consistency of digital identity signals	0–1	Model, Fraud (model and rule), AltData
`device_change_frequency`	Device changes in the period	integer ≥ 0	Model, Fraud (model and rule)
`mobile_wallet_tx_freq`	Wallet transaction frequency	optional, default 0	Model
`gender`, `age_group`	Protected attributes	free text	Fairness audit only; not model inputs
`consent_given`	Consent flag	boolean	Consent gate, decision policy

7. Decision policy and score interpretation

Credit-risk score	Outcome (no override)	Equivalent PD	Meaning
60–100	`APPROVED`	≤ 0.40	Meets the automatic-approval threshold.
45–59.9	`CONDITIONAL_APPROVAL`	0.401–0.55	Moderate risk; text says a lower credit line is recommended.
0–44.9	`DECLINED`	> 0.55	Below the minimum score.

Override order: consent first (`REJECTED_NON_COMPLIANT`), then anomaly or `CRITICAL` fraud (`FLAGGED_FOR_MANUAL_REVIEW`), then score bands. A score of 60 or more is therefore not enough for approval when a fraud rule fires. The review fail-safe can additionally turn `APPROVED` into manual review.

8. Fairness and audit utilities

The same audit logic is reachable three ways: `FairnessAgent.audit()`, `GET /api/v1/fairness-audit` and the MCP tool `run_fairness_audit`. All three read the whole dataset, so they report on the generated data, not on live traffic.

	FairnessAgent (live)	fairness_audit.py (standalone)
Approval rule	score ≥ 60 (PD ≤ 0.40)	PD < 0.45 (score > 55)
Metrics	Approval rate, DIR, parity difference, 80% pass flag	Approval rate, DIR, 80% flag (no parity difference)
Pass flag key	`passes_80_percent_rule`	`is_80_percent_compliant`
Missing reference group	Silently uses rate 1.0	Raises `KeyError`

The two rules disagree, so results differ. With the shipped model every consented applicant is approved under both, giving approval rate 1.0 and ratio 1.0 for every group, which says nothing about fairness. A ratio below 0.80 would be a screening signal, not proof of discrimination; group sizes (for example 44 Non-Binary rows), confidence intervals and intersectional groups also need review.

9. MCP server and tools

`mcp_server.py` builds a `FastMCP` server, loads the artefacts at import time and serves the streamable-HTTP app with uvicorn on `127.0.0.1:8001` (endpoint `/mcp`).

Tool / resource	Behaviour	Consent check
`get_applicant_data(applicant_id)`	Returns the full dataset row (including `gender`, `age_group`, `consent_given`) or 'Applicant not found'.	No
`get_alternative_data(applicant_id)`	Returns AltDataAgent metrics for the stored row.	Yes
`run_risk_prediction(applicant_id)`	Returns XGBoost PD, score and tier.	Yes
`run_fraud_check(applicant_id)`	Returns Isolation Forest and rule results.	Yes
`get_decision_history(applicant_id)`	Returns matching `decision_log.csv` rows.	No
`run_fairness_audit()`	Runs FairnessAgent over the whole dataset.	Filters to consented rows
`underwriting://system` (resource)	Static text describing agents, models and the MCP role.	n/a

FastAPI uses MCP only for the optional lookup in step 3 and for two diagnostic endpoints. The scoring itself always uses the submitted request body, so the MCP record is neither compared with nor substituted for it.

10. REST API reference

Method	Endpoint	Purpose
POST	`/api/v1/underwrite`	Main pipeline. Body: `FullApplicantSchema`. Returns 403 when consent is false.
POST	`/api/v1/event-update`	What-if re-score. Body: `applicant_id`, `event_type`, `metric_updated`, `new_value` (0–1), `applicant_data`. Returns original score, new score, delta and new decision.
GET	`/api/v1/fairness-audit`	Gender and age-group metrics over the dataset.
GET	`/api/v1/decision-history/{applicant_id}`	Logged decisions for one applicant.
GET	`/api/v1/mcp-status`	Tests the MCP connection and lists tool names.
GET	`/api/v1/mcp-applicant/{applicant_id}`	Calls `get_applicant_data` directly; HTTP 500 if the call fails.

`/event-update` reuses `DecisionAgent` with a hard-coded non-anomalous fraud result and consent set to true. It accepts any `metric_updated` string; a name that is not a model feature is silently ignored, and the 0–1 limit on `new_value` makes counts such as `device_change_frequency` impossible to set meaningfully. The README calls this endpoint `/update-behavior`; the code uses `/event-update`. `/mcp-status` returns a hard-coded version string ('1.30.0').

11. Dashboard

`dashboard_flask.py` serves one HTML page (Tailwind via CDN, Inter and JetBrains Mono fonts) on port 5000 with `debug=True`. It embeds four profiles: `APP_CUSTOM`, `APP_1042` (gig worker), `APP_2091` (salaried) and `APP_3310` (suspicious-pattern test case). The page offers a consent toggle, numeric inputs and sliders for six alternative signals, then posts the profile to `http://localhost:8000/api/v1/underwrite` and renders the decision, score, default probability, fraud assessment, SHAP drivers and letter. An error panel appears if the backend cannot be reached. It does not call `/event-update`; the base URL is hard-coded.

12. Running the project

Run everything from the project root; paths such as `underwriting_artifacts.pkl` are relative.

```
pip install fastapi uvicorn pandas numpy scikit-learn xgboost shap joblib pydantic
requests mcp flask
python generate_dataset.py
python train_model.py
ollama pull llama3.2          # optional
python mcp_server.py          # terminal 1, port 8001
python app.py                 # terminal 2, port 8000 (reload=True)
python dashboard_flask.py     # terminal 3, port 5000
```

Open `http://127.0.0.1:8000/docs` for Swagger UI and `http://localhost:5000` for the dashboard. `requirements.txt` is out of step with the code: it omits `mcp` and `flask`, and lists `fairlearn`, `streamlit` and `plotly`, none of which the code imports. The README badges (Python 3.11, FastAPI 0.115) and its folder tree (four agents, no MCP server) are also incomplete.

13. Verified findings and recommended fixes

Priority	Finding	Recommended fix
High	**Degenerate model.** One default in 2,000 rows; every input scores 99.9; SHAP and fairness outputs are uninformative.	Add an intercept to the latent formula (or lower the cut-off) to reach a realistic default rate, regenerate, retrain, and check ROC-AUC, calibration and score spread before demos.
High	**Consent is one Boolean.** Not a compliance framework.	Store a consent record (purpose, categories, timestamp, notice version, withdrawal) and obtain legal review before claiming compliance.
High	**Synthetic data only.** Metrics reflect the generator.	Do not use for real lending; validate on lawful, representative data with out-of-time tests.
High	**Letter and decision can disagree.** After the fail-safe, the letter is not regenerated.	Generate the letter after the final decision, or regenerate on override.
Medium	**MCP result is not the inference source.** IDs and data can mismatch.	Choose one authoritative source; reject mismatches; verify identity and consent.
Medium	**Zero-fill of missing features.** Distorts predictions.	Require fields or use training-fitted imputation with missingness flags.

Priority	Finding	Recommended fix
Medium	**Partial fail-safe.** Only `APPROVED` is downgraded.	Route every failed review to manual review.
Medium	**Fairness inconsistencies.** Different thresholds, missing-group handling and key names.	Share one policy function; guard empty data, absent reference groups and tiny groups.
Medium	**Event endpoint bypasses policy.** Fixed fraud result, consent true, unvalidated metric name.	Reuse real consent and fraud logic, whitelist metrics, or label it a what-if simulator.
Medium	**MCP exposes protected attributes and skips consent** in `get_applicant_data`.	Add a consent check and return only permitted fields.
Medium	**Fallback letter inaccuracies.** Labels all non-approvals 'DECLINED' and asserts a 30-day right.	Write one template per outcome and verify the legal wording.
Low	**Empty SHAP list** would break the fallback letter; **AltDataAgent** assumes valid ranges.	Add guards and clamp or validate inputs.
Low	**Debug prints** write applicant features to the console.	Remove or redact; use structured logging.
Low	**Deployment settings.** CORS `*`, host `0.0.0.0`, `reload=True`, Flask `debug=True`, no authentication.	Restrict origins, add auth, TLS, rate limits and secrets management.
Low	**CSV decision log.** No locking, access control or tamper evidence.	Use a transactional database with protected audit storage.
Low	**Housekeeping.** `requirements.txt` and README drift (Section 12).	Align dependencies, endpoint names and the folder tree with the code.

14. Security, privacy and responsible use

- Collect only data that is necessary, permitted and disclosed to the applicant.
- Record consent and withdrawal durably; stop processing when consent is withdrawn.
- Protect identifiers and behavioural signals with encryption, role-based access and retention limits.
- Keep protected attributes out of model inputs, but keep auditing outcomes for group disparities.
- Provide human review and a clear explanation process for adverse or uncertain outcomes.
- Treat an anomaly score as a reason to investigate, never as proof of fraud.
- Do not use the synthetic-data model for credit decisions without domain, legal, privacy, security and model-risk validation.

15. Glossary

Term	Explanation
Underwriting	Assessing the risk of extending credit to an applicant.
PD	Probability of default, 0 to 1.

Term	Explanation
Credit-risk score	(1 − PD) × 100; higher is better here.
XGBoost	Gradient-boosted decision trees, used for default classification.
Isolation Forest	Unsupervised anomaly detector that isolates unusual rows.
SHAP	Method that attributes a prediction to individual features.
DIR / 80% rule	Subgroup approval rate divided by the reference group's rate; 0.80 is a common screening threshold.
MCP	Model Context Protocol, a standard way to expose tools and resources to AI clients.
Ollama	Local runner for language models; drafts the decision letter.
DPDP Act 2023	India's Digital Personal Data Protection Act.

16. Conclusion

The project shows a clear multi-agent design: consent gate, alternative-data summaries, anomaly detection, risk prediction, SHAP explanations, policy decisions, self-review, fairness auditing, what-if updates and MCP access, each in a small, readable class. The immediate priority is to repair the training data so the model produces meaningful scores. After that, align the thresholds, harden consent and data governance, fix the edge cases in Section 13 and validate on representative real data before any operational use.